Fraud detection mit Stream Analytics

BATbern43 – 28.6.2019

PostFinance

ELCA

Vorstellung & Agenda

Über uns

Matej Ilicic hat 2011 den Master of Science in Software Engineering an der Universität Zagreb abgeschlossen, war 5 Jahre als Software-Engineer tätig, und ist in den letzten 2,5 Jahren bei der PostFinance als Solution Architekt hauptsächlich in Compliance & Risk Bereich tätig. (Linkedin)

Zacharias Kull hat Software Entwicklung studiert und ist Datenexperte bei ELCA. Er ist fasziniert von den Möglichkeiten neuer Technologien wie Kafka und Spark und wie dadurch verschiedene Disziplinen wie Batch, Streaming und DevOps zusammenkommen. (Linkedin)

Agenda

- Hintergrund Betrugserkennung
- Betrugserkennung bei der PostFinance
- Technologiewahl
- Lösungsdesign
- Projekt und Lessons Learned

Definition vom Betrug/Fraud

Gemäss Wikipedia, Betrug ist ein strafrechtliches Vermögensdelikt, bei dem der Täter in der Absicht rechtswidriger Bereicherung das Opfer durch Vorspiegelung oder Unterdrückung von Tatsachen gezielt so täuscht, dass es sich selbst oder einen Dritten am Vermögen schädigt und damit materiellen Schaden zufügt.

Betrugskategorien:

Extern

- Kreditkartebetrug
- Onlinebetrug
- Transaktionsbetrug
- Claims Fraud → Versicherungen
- Andere

Operational Intelligence

Intern

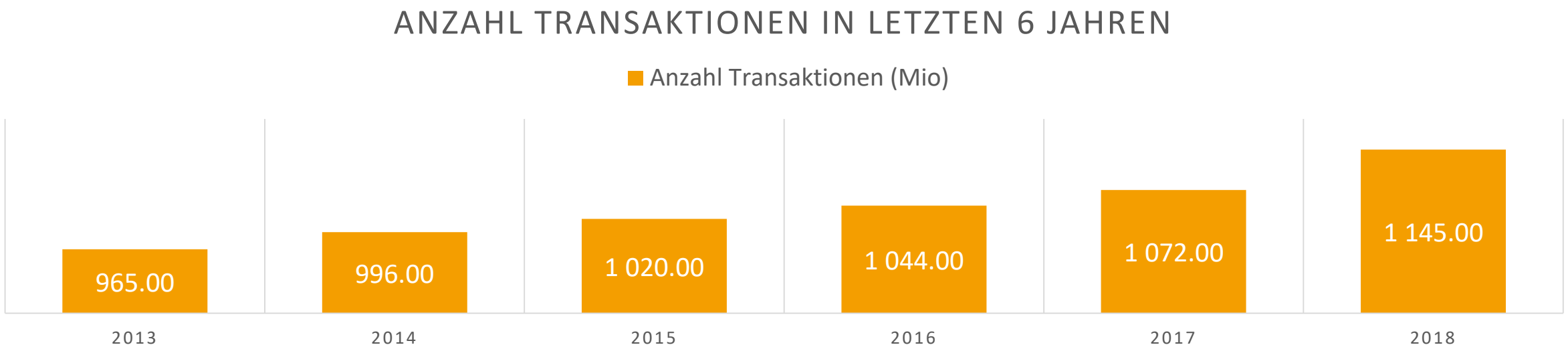
- Betrug mit Finanzberichten
- Korruption
- Veruntreuung vom Vermögen

Betrugserkennungssystem

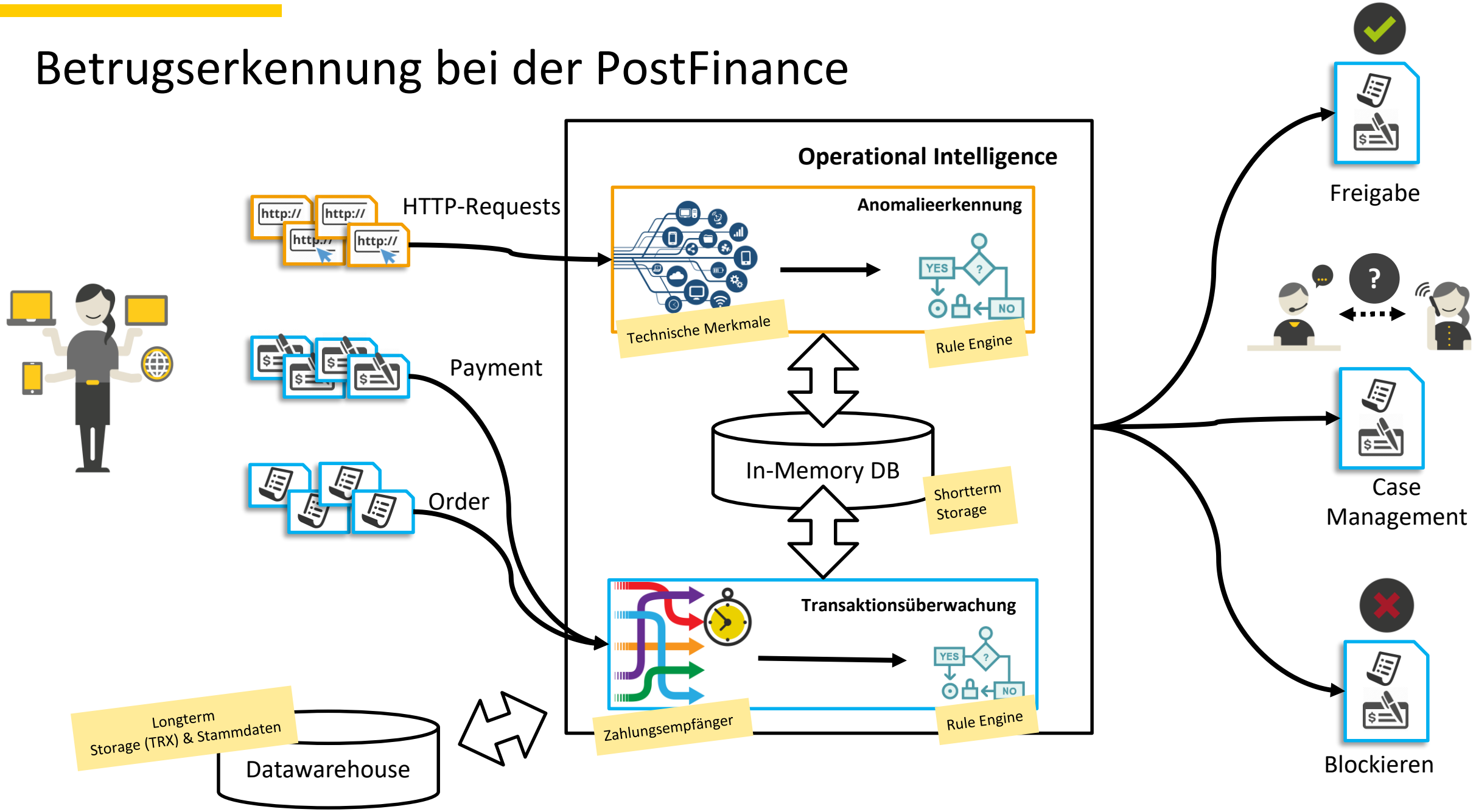
- Automatisiert und stärkt die Monitoring-Aktivitäten eines Unternehmens
- Monitoring ist basiert auf den Daten von verschiedenen Datenquellen/Sensoren (z.B. Behavior Analytics, Geolocation, Transaktionen, Kartengeldtransaktionen, Anti Money Laundering (AML) Applikation…)
- Erkennungs- und Präventionsprozess sind basiert auf den Regeln, prädiktiven Modellen oder gemischt
- Ermöglicht ein Fokus der Ressourcen auf die Analyse der Betrugsfälle anstatt auf die manuelle Erkennung

PostFinance in Facts

1. 3’325 Mitarbeiter in 2018 davon ca. 500 in Informatik
2. 2’857 Mio Kunden in 2018
3. 4’503 Mio Konten
4. Zwischen 1,5 und 2,5 Mio Transaktionen täglich und 0,1% davon landen in Case Management
5. 497’594 Transaktionen pro Stunde am 29.5.2019 12:00
6. Überwachung von ca. 5000 gleichzeitige Sessions in EFinance
7. Daraus ca. 24’000’000 Mio Ereignisse pro Tag



Betrugserkennung bei der PostFinance



Operational Intelligence - Aufgaben

- Korrelation von Events
- Real-time Monitoring
- Real-time Situationserkennung
- Auslösen von Aktionen (Session Terminieren, Read-Only Modus, Alarmieren vom Online-Sicherheit)

Operational Intelligence - Betrugserkennungsprozess

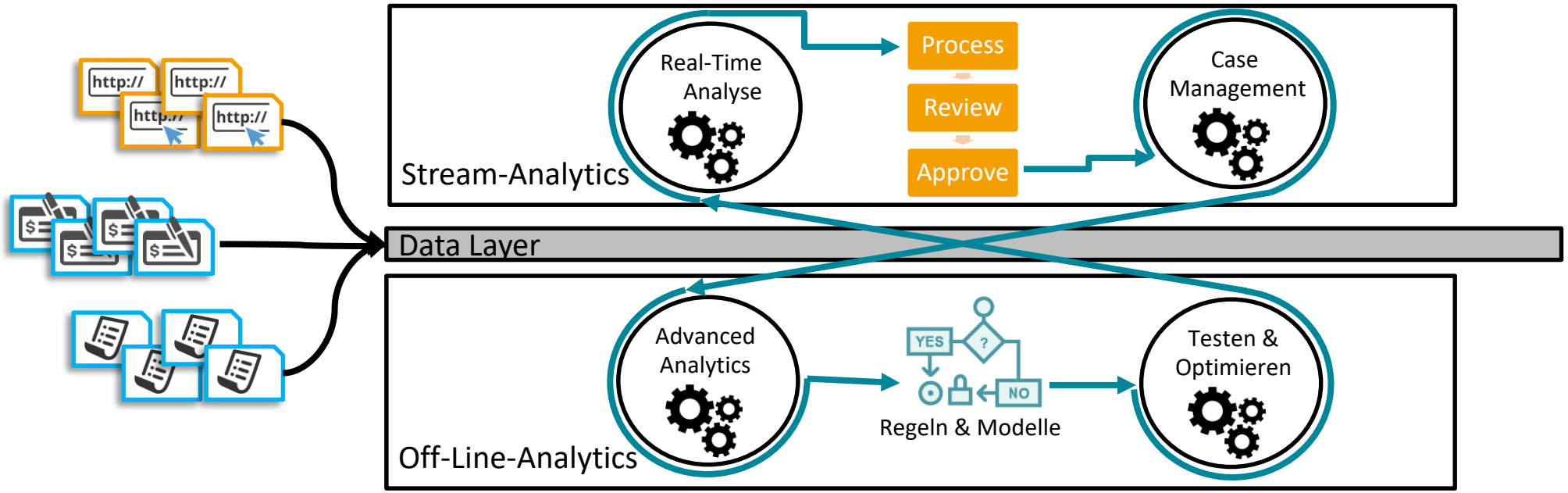
Heutiger Betrugserkennungsprozess basiert auf Regeln bzw.

- Experten Wissen
- Trial and Error
- Domänen Know How
- Intuition

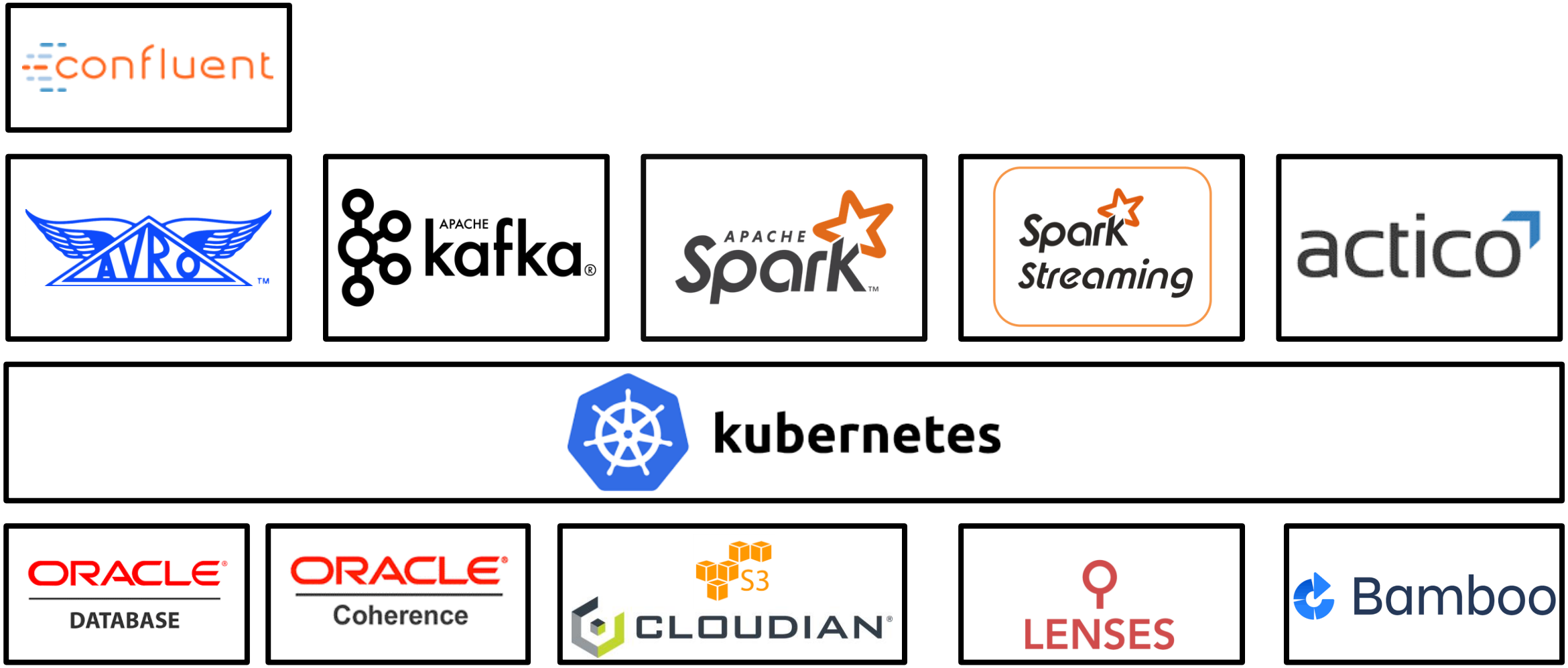
Operational Intelligence – Optimierungen & Weiterentwicklung

Optimierungen des heutigen Betrugserkennungsprozesses

1. Simulationen mit den Regeln/Modellen und Live-Daten→ Apache Kafka als Grundlage
2. Einbau der zusätzlichen Sensoren (DeviceTracking, Behavior Tracking…)
3. Verbindung und Benutzung der Daten aus anderen Systemen (Kartengeld, AML, Embargo-Listen…)
4. Optimierung der Betrugserkennung mit Machine Learning (Selflearning Feedback Loop)



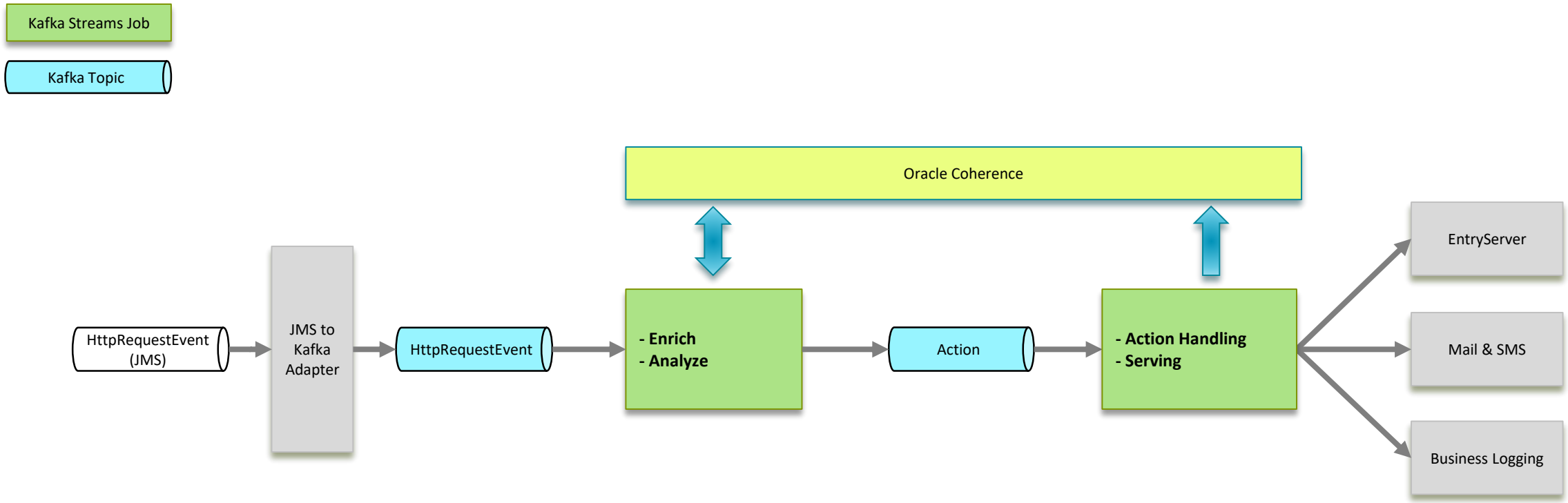
Technologien



Kafka Streams vs Spark Streaming vs Flink

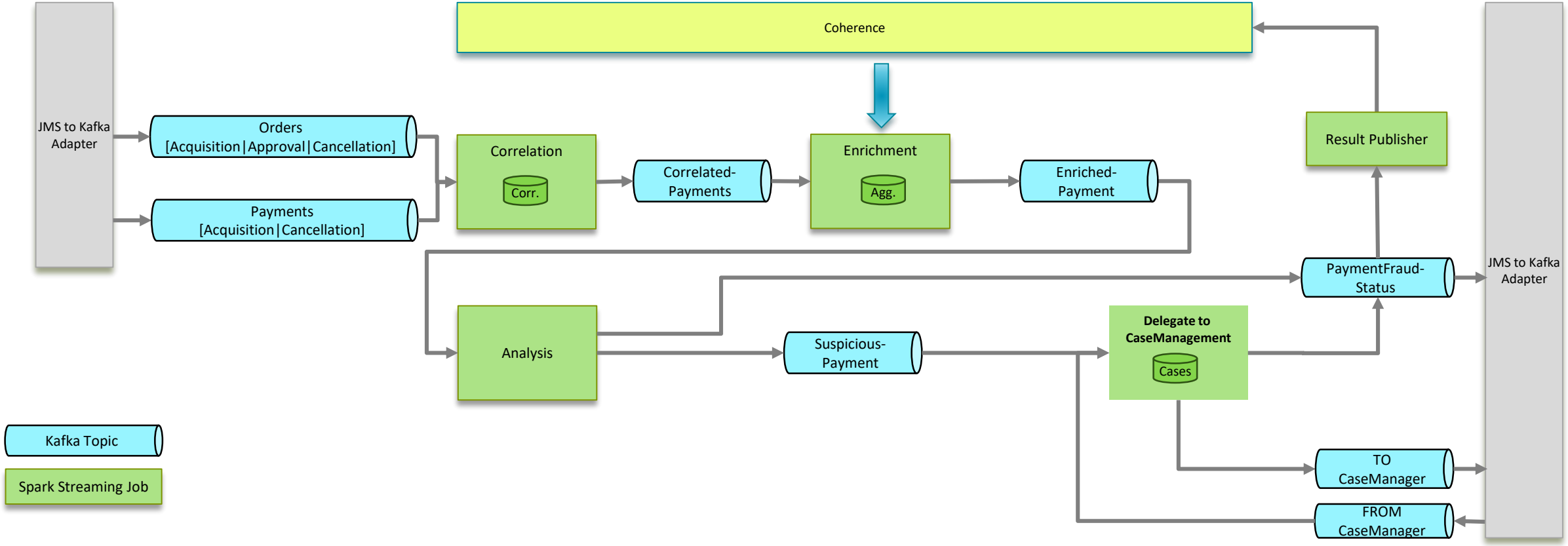
	Kafka Streams	Apache Spark Streaming	Apache Flink
Latenz	Tief (ms)	Hoch (sec)	Tief (ms)
State Management	Statefull	Statefull	Statefull
Know-How auf dem Arbeitsmarkt	😐	😐	☹
Deployment	Container (wie die Applikation)	Container	Container
Life cycle	Wie die Applikation	Spark Job auf dem Spark-Cluster	Flink Job auf dem Spark-Cluster
Streaming-Typ	Native Streaming	Micro-Batching	Native-Streaming
Durchsatz	Mittelmässig	Hoch	Hoch
Verwendet in	**OI - Anomalieerkennung**	**OI - Transaktionsüberwachung**	**X**

Lösungsdesign «Anomalieerkennung»



NFR-s:
NFR-s: Target 99th percentile latency of 500 ms
Resilience / SLA: Highly available (99.5%)

Lösungsdesign «Transaktionsüberwachung»



NFR-s

- Max. processing time: 2.5min for Express payments, 15min for Normal Payments
- Volume: 1 Mio Payments/h
- Komplete und korrekte Prozessierung (at-least-once)
- Resilience / SLA: Highly available (99.5%)

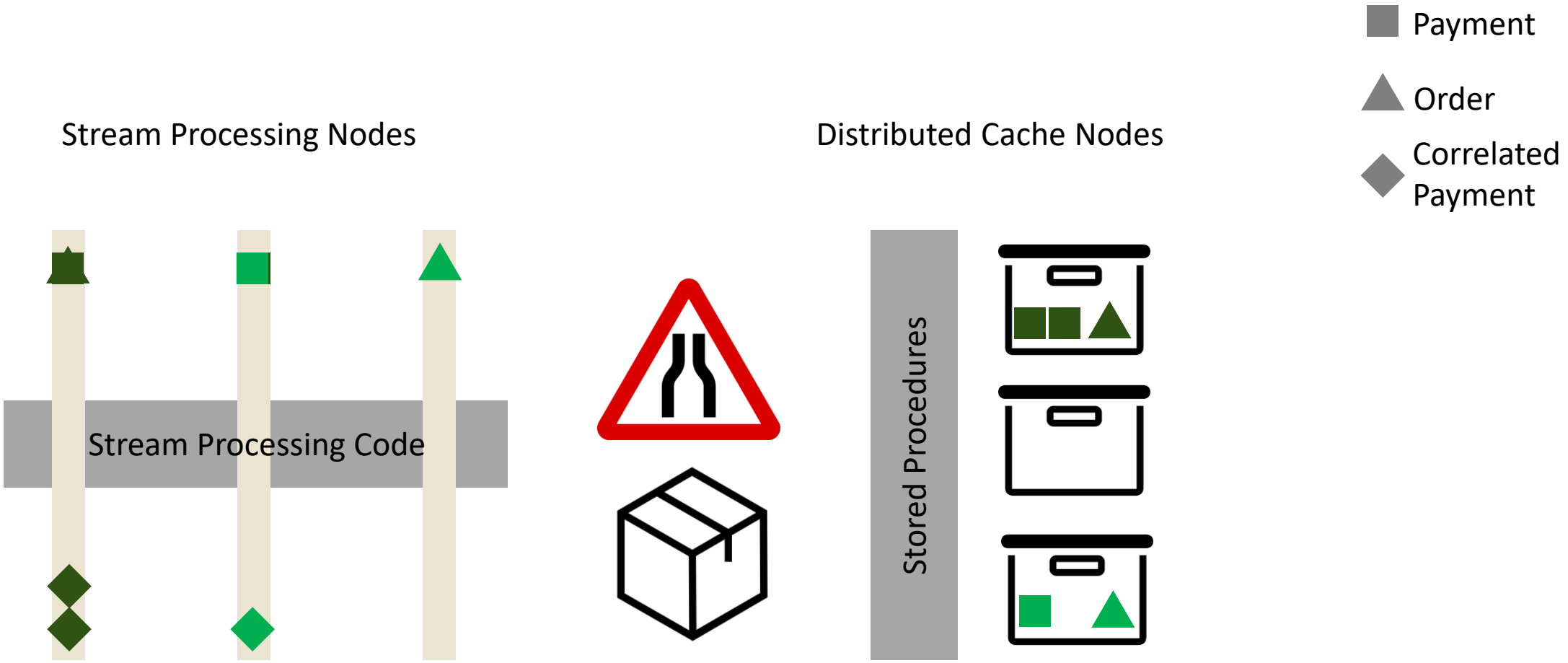
Komplexe Stream Korrelation

Die zwei Streams Order und Payment sollen korreliert (gejoined) werden. Jedes Payment soll mit dem zugehörigen Order zusammengesetzt dem nächsten Aufbereitungsschritt übergeben werden.

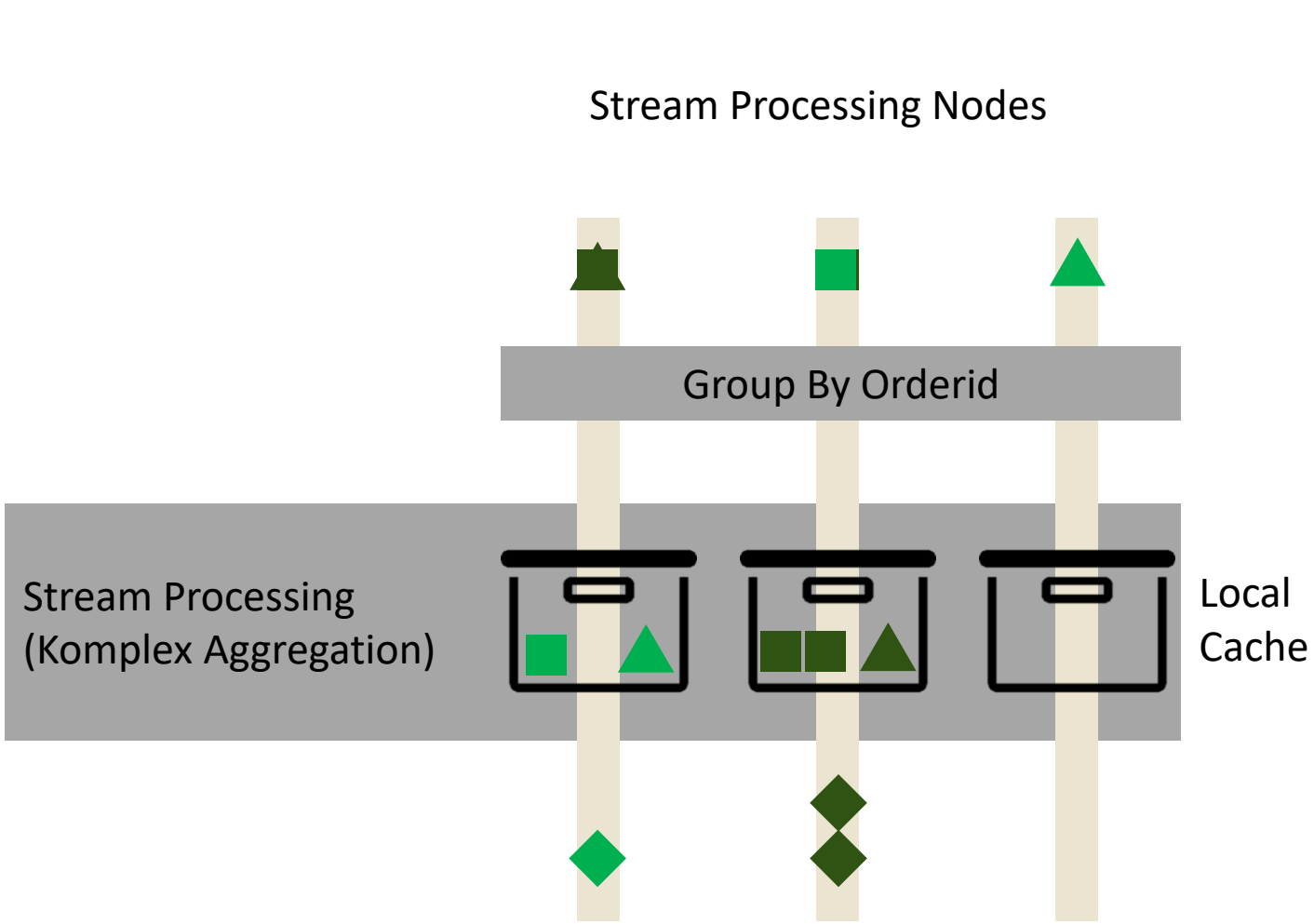
Anforderungen:

- Die Order oder die zugehörigen Payments können zuerst sein
- Eine Order kann bis zu 100’000 Payments haben (normalerweise aber 1:1)
- Order und Payment können separat abgebrochen werden. Wird die Order abgebrochen müssen alle bisher weitergeleiteten Payments nochmals als abgebrochen weitergeleitet werden
- Nicht genehmigte Orders müssen bis 2 Jahre vorgehalten werden
- Änderungen an einem Payment sollen bis zu 24h nach kompletter Verarbeitung der Order möglich sein

Bisherige Implementation der Korrelation

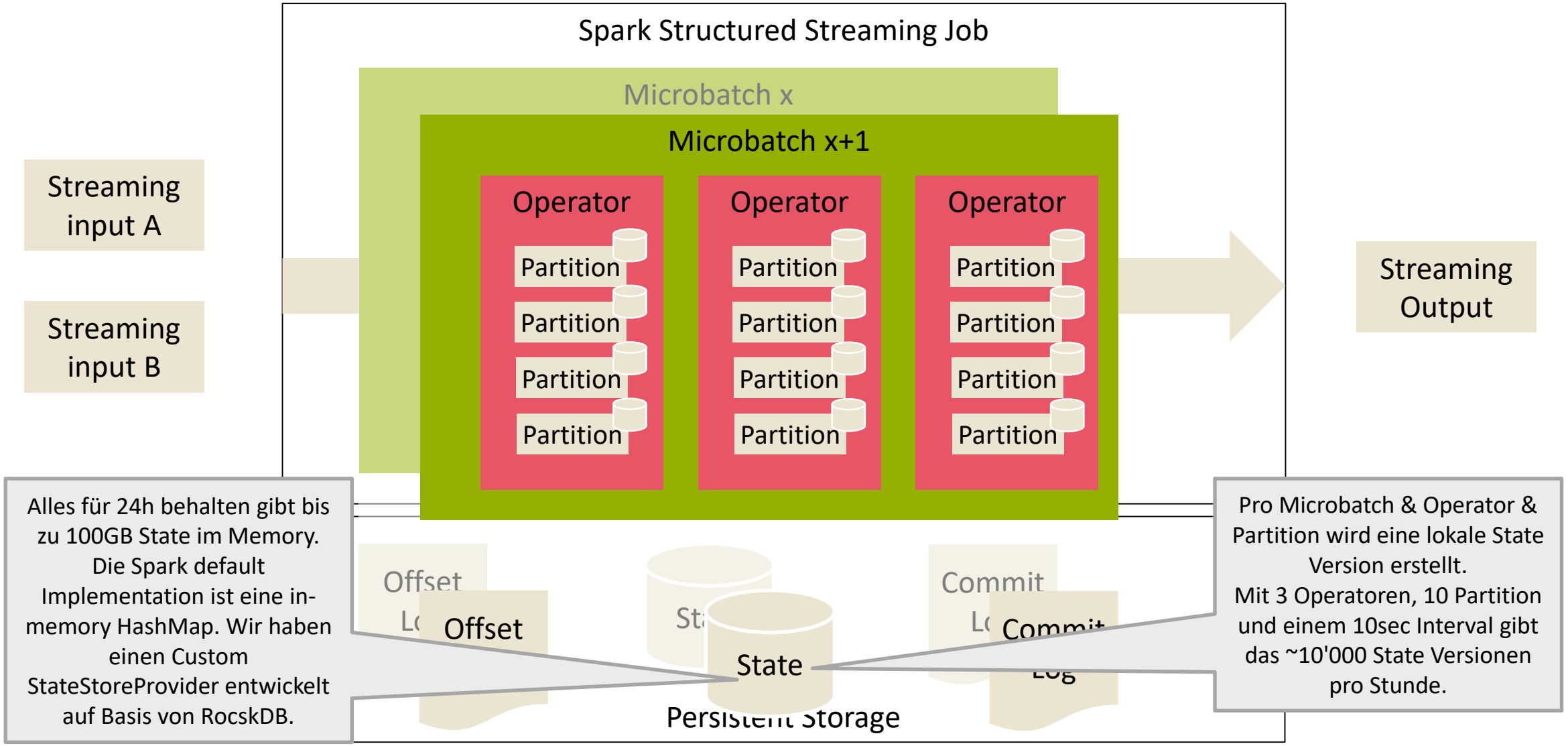


Neue Implementation der Korrelation als «Komplexe Aggregation»



Die einfachere Implementation erlaubte weitere innovative Anforderungen in der Correlation umzusetzen, welche bisher zu komplex waren.

Spark Microbatches: at-least-once Garantie und State



Resilience: Mehrere Stufen der Fehlerbehandlung

Level	Error	Mögliche Ursache	Resilience Implementation
User Code	Exception in Src/Tgt/Custom code von Spark Task	• S3 Up/Download failed • Kafka Connection errors • Logic error	• Tune resilience settings (S3, Kafka) • Implement custom error handling
Spark Framework	Spark Task failes	• Uncatched Exceptions • OutOfMemory • Executor Killed (e.g. Chaos Monkey)	• Spark Task Retry
Kubernetes	Spark Job (Driver) failes	• Task Retries failed • OutOfMemory • Driver Killed (e.g. Chaos Monkey)	• Kubernetes Statefulset restarts Spark Job

Anspruchsvolles Spark Performance Tuning

Ziel	Parameters
Throughput	+ Microbatch size + Number of Spark Partitions + Number of processing Cores - Microbatch interval
Latency	- Microbatch size - Number of Spark Partitions + Number of processing Cores - Microbatch interval
Memory	- Microbatch size + Number of Spark Partitions - Number of processing Cores

Parameter	Einfluss
Microbatch size	+ Throughput + Latency + Memory
Number of Spark Partitions	+ Throughput - Latency + Memory
Number of processing Cores	+ Throughput - Latency + Memory
Microbatch interval	- Throughput - Latency

Aktuelles Sizing / Performance

Kafka:
- 3 Broker mit je 4GB Ram, 2 Cores, 1TB iSCSI Disk
- 15 Partitionen je Topic

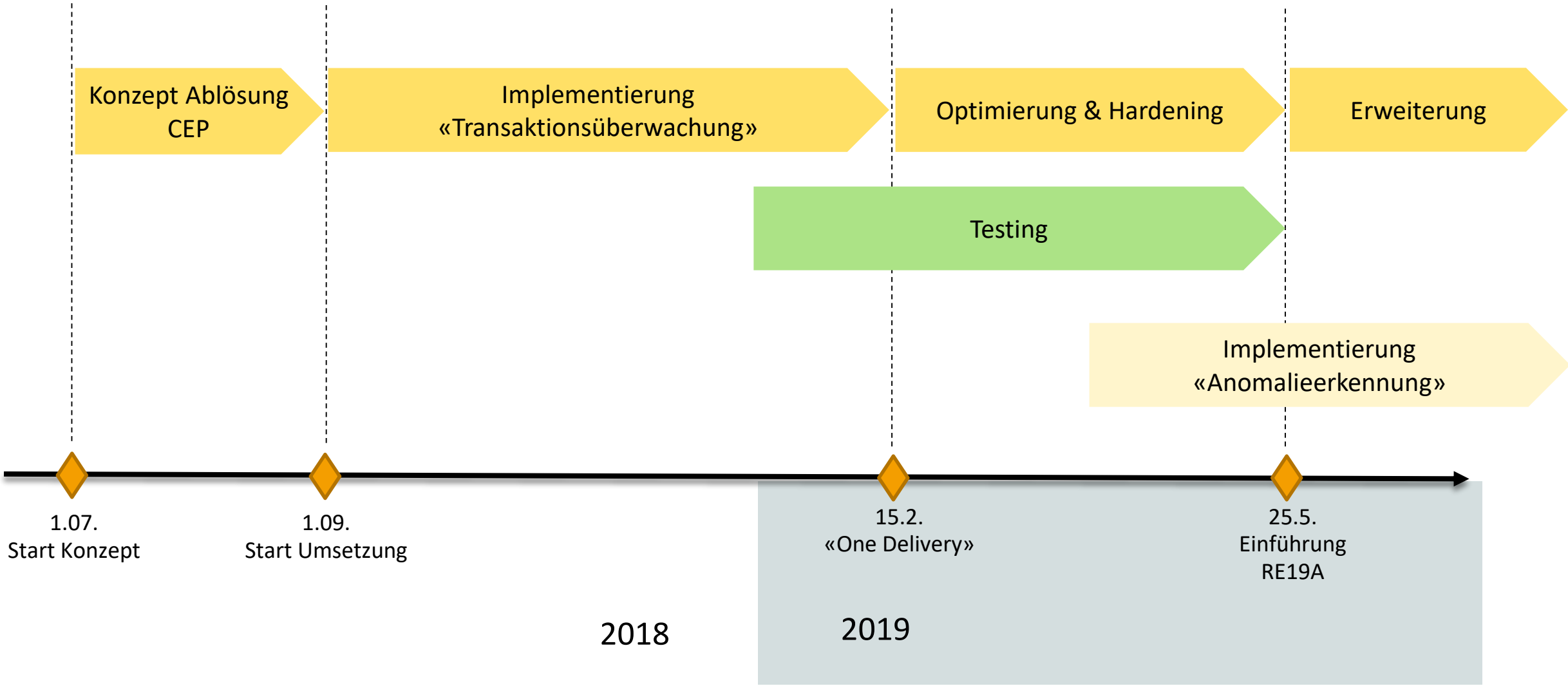
Spark (für die 3 Haupt-Streaming-Queries):
- Driver mit 10GB Ram, 1 Core
- 3 Executors mit je 16GB Ram, 6 Cores, 10GB Disk
- 10 Partitionen
- 30’000 Messages pro Microbatch

  → Erreichtes Microbatch Interval >10s (je nach Last)

  → Performance erfüllt die NFR’s (1Mio/h)

  Optimierungspotential ist noch nicht ausgeschöpft

Projektplanung «Transaktionsüberwachung»



Was wir erreicht haben…

- Infrastruktur für Spark auf Kubernetes aufgebaut und Pilot-Projekt darauf umgesetzt (inkl. Apache Hadoop OpenSource Bugfix)
- Stage-of-the-art Messagingarchitektur mit Kafka und Schemaregistry sowie typensicheren Streaming Pipelines mit Spark implementiert (inkl. avro4s OpenSource Library Improvement).
- Komplexe Stream Correlation horizontal skalierbar implementiert
- Komplexe Stream Correlation mit «Test Driven Development» umgesetzt
- Automatisiertes Deployment mit Atlassian Bamboo umgesetzt
- Hohe nicht funktionale Anforderungen an Performance, Stabilität, Vollständigkeit und Betreibbarkeit umgesetzt.
- Go-Live mit RE19A Ende Mai 2019

Lessons Learned…

- Aufbau der Infrastruktur brauchte viel Zeit
  → tiefes Technologieverständnis hilft viel
- Rückstand beim Testing hat fast den Release verhindert
- Aufgrund der vielen neuen Technologien hatten wir eine lange Optimierung-, Stabilisierung- und Bugfixingphase bis kurz vor dem Release
  → weniger neue Technologie wäre vielleicht mehr
- Optimierung und Bugfixing in solchen verteilten Systemen ist anspruchsvoll
  → ausführliches Logging auf Splunk hilft viel
- AWS S3 ist nur “eventual consistent”, Cloudian S3 bietet mehr, aber…
  → Konsistenzfehler verursachen immer noch Abbrüche
- Wären Order und Payment nicht getrennt, wäre es einiges einfacher

Fragen…